Name of Subject & Class: Programming Skill Development Lab (214455) SE(IT)
Examination Scheme: PR: 25Marks TW: 25Marks

Course Outcomes:
On completion of this course student will be able to --
CO1: Apply concepts related to embedded C programming.
CO2: Develop and Execute embedded C program to perform array addition, block transfer, sorting operations
CO3: Perform interfacing of real-world input and output devices to PIC18FXXX microcontroller.
CO4: Use source prototype platform like Raspberry-Pi/Beagle board/Arduino.

List of Experiment

Sr.No.	Name Of Experiment
Group A (Any Three)	
Mapping of Course Outcomes for Group A -- CO1 , CO2	
1	**Study of Embedded C programming language (Overview, syntax, One simple program like addition of two numbers).**
2	**Write an Embedded C program to add array of n numbers.**
3	**Write an Embedded C program to transfer elements from one location to another for following: i) Internal to internal memory transfer ii) Internal to external memory transfer**
4	**Write an Embedded C menu driven program for : i) Multiply 8 bit number by 8 bit number ii) Divide 8 bit number by 8 bit number**
5	**Write an Embedded C program for sorting the numbers in ascending and descending order.**
Group B (Any Three)	
Mapping of Course Outcomes for Group B -- CO3	
6	**Write an Embedded C program to interface PIC 18FXXX with LED & blinking it using specified delay.**
7	**Write an Embedded C program for Timer programming ISR based buzzer on/off.**
8	**Write an Embedded C program for External interrupt input switch press, output at relay.**
9	**Write an Embedded C program for LCD interfacing with PIC 18FXXX.**
Group C (Any two)	
Mapping of Course Outcomes for Group C -- CO3	
10	**Write an Embedded C program for Generating PWM signal for servo motor/DC motor.**
11	**Write an Embedded C program for PC to PC serial communication using UART.**
12	**Write an Embedded C program for Temperature sensor interfacing using ADC & display on LCD.**
Group D	
13	**Study of Arduino board and understand the OS installation process on Raspberry-pi.**
14	**Write simple program using Open source prototype platform like Raspberry-Pi/Beagle board/Arduino for digital read/write using LED and switch Analog read/write using sensor and actuators.**

Group A

Experiment No. 1

Title : -Study of Embedded C programming language (Overview, syntax, One simple program like addition of two numbers).

Aim : To Study of Embedded C programming language
Objectives : 1. To understand the Embedded C programming for PIC18FXXX
Outcome : After performing this experiment student will be able to –

1. Apply concepts related to embedded C programming.
2. able to write embedded C language program with appropriate arithmetical operators.

Apparatus/ Software required: MPLAB X Programming IDE
Theory:
What is an Embedded System?
An Embedded System can be best described as a system which has both the hardware and software and is designed to do a specific task. A good example for an Embedded System, which many households have, is a Washing Machine.
Embedded Systems can not only be stand-alone devices like Washing Machines but also be a part of a much larger system. An example for this is a Car. A modern day Car has several individual embedded systems that perform their specific tasks with the aim of making a smooth and safe journey.
Embedded C :
Earlier, many embedded applications were developed using assembly level programming. However, they did not provide portability. This disadvantage was overcome by the advent of various high-level languages like C, Pascal, and COBOL. However, it was the C language that got extensive acceptance for embedded systems, and it continues to do so. The C code written is more reliable, scalable, and portable; and in fact, much easier to understand. Embedded C Programming is the soul of the processor functioning inside each and every embedded system we come across in our daily life, such as mobile phones, washing machines, and digital cameras. Each processor is associated with embedded software. The first and foremost thing is the embedded software that decides to function of the embedded system. Embedded C language is most frequently used to program the microcontroller.
What is C Language?
C language was developed by Dennis Ritchie in 1969. It is a collection of one or more functions, and every function is a collection of statements performing a specific task.
C language is a middle-level language as it supports high-level applications and low-level applications. Before going into the details of embedded C programming, we should know about RAM memory organization.

The main features of the C language include the following.

- C language is software designed with different keywords, data types, variables, constants, etc.
- Embedded C is a generic term given to a programming language written in C, which is associated with a particular hardware architecture.
- Embedded C is an extension to the C language with some additional header files. These header files may change from controller to controller.
- The microcontroller 8051 #include<reg51.h> is used.

What is an Embedded C Programming:

In every embedded system based projects, Embedded C programming plays a key role to make the microcontroller run & perform the preferred actions. At present, we normally utilize several electronic devices like mobile phones, washing machines, security systems, refrigerators, digital cameras, etc. The controlling of these embedded devices can be done with the help of an embedded C program. For example in a digital camera, if we press a camera button to capture a photo then the microcontroller will execute the required function to click the image as well as to store it.

Embedded C programming builds with a set of functions where every function is a set of statements that are utilized to execute some particular tasks. Both the embedded C and C languages are the same and implemented through some fundamental elements like a variable, character set, keywords, data types, declaration of variables, expressions, statements. All these elements play a key role while writing an embedded C program.

The embedded system designers must know about the hardware architecture to write programs. These programs play a prominent role in monitoring and controlling external devices. They also directly operate and use the internal architecture of the microcontroller, such as interrupt handling, timers, serial communication, and other available features.

Embedded System Programming

The designing of an embedded system can be done using Hardware & Software. For instance, in a simple embedded system, the processor is the main module that works like the heart of the system. Here a processor is nothing but a microprocessor, DSP, microcontroller, CPLD & FPGA. All these processors are programmable so that it defines the working of the device.

An Embedded system program allows the hardware to check the inputs & control outputs accordingly. In this procedure, the embedded program may have to control the internal architecture of the processor directly like Timers, Interrupt Handling, I/O Ports, serial communications interface, etc.So embedded system programming is very important to the processor. There are different programming languages are available for embedded systems such as C, C++, assembly language, JAVA, JAVA script, visual basic, etc. So this programming language plays a key role while making an embedded system but choosing the language is very essential.

Steps to Build an Embedded C Program

There are different steps involved in designing an embedded c program like the following.

- Comments
- Directives of Processor
- Configuration of Port
- Global variables
- Core Function/Main Function
- Declaration of Variable
- The logic of the Program

Comments

In programming languages, comments are very essential to describe the program's function. The code of the comments is non-executable but used to provide program documentation. To understand the function of the program, this will make a simple method to understand the function of the program. In embedded C, comments are available in two types namely single line and mainline comment.In an embedded C programming language, we can place comments in our code which helps the reader to understand the code easily.

C=a+b; /* add two variables whose value is stored in another variable C*/

Single Line Comment

Generally, for the programming languages, single-line comments are very useful to clarify a fraction of the program. These comments begin with a double slash (//) and it can be located anywhere within the programming language. By using this, the whole line can be ignored within a program.

Multi-Line Comment

Multi-line comments begin with a single slash (/) & an asterisk (/*) in the programming languages which explains a block of code. These types of comments can be arranged anywhere within the programming language and mainly used to ignore a whole block of code within a program.

Directives of Processor

The lines included within the program code are called preprocessor directives which can be followed through a hash symbol (#). These lines are the preprocessor directives but not programmed statements.
The code can be examined through a preprocessor before real code compilation starts & resolves these directives before generating a code through regular statements. There are several special preprocessor directives are available although two directives are extremely helpful within the programming language like the following.

```
#include
#include<reg51.h>
Sbit LED = P2^3;
Main();
{
LED = 0x0ff
Delay();
LED=0x00;
}
#define
#include<reg51.h>
#define LED P0
Main();
{
LED = 0x0ff
Delay();
LED=0x00;
}
```

In the above program, the #include directive is generally used to comprise standard libraries like study and. h is used to allow I/O functions using the library of ‘C’. The #define directive usually used to describe the series of variables & allocates the values by executing the process within a particular instruction like macros.

Configuration of Port

The microcontroller includes several ports where every port has different pins. These pins can be used for controlling the interfacing devices. The declaration of these pins can be done within a program with the help of keywords. The keywords in the embedded c program are standard as well as predefined like a bit, sbit, SFR which are used to state the bits & single pin within a program.

There are certain words that are reserved for doing specific tasks. These words are known as keywords. They are standard and predefined in the Embedded C. Keywords are always written in lowercase. These keywords must be defined before writing the main program. The main functions of the keywords include the following.

```
#include< >
Sbit a = P 2^2;
SFR 0x00 = PoRT0;
Bit C;
main()
{
……..
……..
}
```

sbit

This is one kind of data type, used to access a single bit within an SFR register.
The syntax for this data type is : sbit variable name = SFR bit ;

Example: sbit a=P2^1;
If we assign p2.1 as ‘a’ variable, then we can use ‘a’ instead of p2.1 anywhere in the program, which reduces the complexity of the program.

Bit

This type of data type is mainly used for allowing the bit addressable memory of random access memory like 20h to 2fh.
The syntax of this data type is : name of bit variable;
Example: bit c;
It is a bit series setting within a small data region that is mainly used with the help of a program to memorize something.

SFR

This kind of data type is used to obtain the peripheral ports of the SFR register through an additional name. So, the declaration of all the SFR registers can be done in capital letters.
The syntax of this data type is: SFR variable name = SFR address for SFR register;
Example: SFR port0 = 0×80;

If we allocate 0×80 like ‘port0’, after that we can utilize 0×80 in place of port0 wherever in the programming language to decrease the difficulty of the program.

SFR Register

The SFR stands for Special Function Register. In 8051 microcontroller, it includes the RAM memory with 256 bytes, which is divided into two main elements: the first element of 128 bytes is mainly utilized for storing the data whereas the other element of 128 bytes is mainly utilized to SFR registers. All the peripheral devices such as timers, counters & I/O ports are stored within the SFR register & every element includes a single address.

Global Variables

When the variable is declared before the key function is known as the global variable. This variable can be allowed on any function within the program. The global variable's life span mainly depends on the programming until it reaches an end.

```
#include<reg51.h>
Unsigned int a, c =10;
Main()
{
……
……..
}
```

Core Function / Main Function

The main function is a central part while executing any program and it begins with the main function simply. Each program utilizes simply one major function since if the program includes above one major function, next the compiler will be confused in begin the execution of the program.

```
#include<reg51.h>
Main()
{
……
……..
}
```

Declaration of Variable

The name like the variable is used for storing the values but this variable should be first declared before utilized within the program. The variable declaration states its name as well as a data type. Here, the data type is nothing but the representation of storage data. In embedded C programming, it uses four fundamental data types like integer, float, character for storing the data within the memory. The data type size, as well as range, can be defined depending on the compiler.

The data type refers to an extensive system for declaring variables of different types like integer, character, float, etc. The embedded C software uses four data types that are used to store data in memory.

The ‘char’ is used to store any single character; ‘int’ is used to store integer value, and ‘float’ is used to store any precision floating-point value. The size and range of different data types on a 32-bit machine are given in the following table. The size and range may vary on machines with different word sizes.

- The char/signed char data type size is 1 byte and its range is from -128 to +128
- The unsigned char data type size is 1 byte and its range is from 0 to 255
- Int/signed int data type size is 2 byte and its range is from -32768 to 32767
- Unsigned int data type size is 2 byte and its range is from 0 to 65535

```
Main();
{
Unsigned int a,b,c;
}
```

The Structure of an Embedded C Program is shown below.

- comments
- preprocessor directives
- global variables
- main() function

{

- local variables
- statements
- ……..
- ……..

}

- fun(1)

{

- local variables
- statements
- ……..
- ……..

}

The logic of the Program

The logic of the program is a plan of the lane that appears in the theory behind & predictable outputs of actions of the program. It explains the statement otherwise theory regarding why the embedded program will work and shows the recognized effects of actions otherwise resources.

```
Main
{
LED = 0x0f;
delay(100);
LED = 0x00;
delay(100);
}
```

Main Factors of Embedded C Program

The main factors to be considered while choosing the programming language for developing an embedded system include the following.

Program Size

Every programming language occupies some memory where embedded processor like microcontroller includes an extremely less amount of random access memory.

Speed of the Program

The programming language should be very fast, so should run as quickly as possible. The speed of embedded hardware should not be reduced because of the slow-running software.

Portability

For the different embedded processors, the compilation of similar programs can be done.

- Simple Implementation
- Simple Maintenance
- Readability

C Language	Embedded C Language
Generally, this language is used to develop desktop-based applications	Embedded C language is used to develop microcontroller-based applications.
C language is not an extension to any programming language, but a general-purpose programming language	Embedded C is an extension to the C programming language including different features such as addressing I/O, fixed-point arithmetic, multiple-memory addressing, etc.
It processes native development in nature	It processes cross development in nature
It is independent for hardware architecture	It depends on the hardware architecture of the microcontroller & other devices
The compilers of C language depends on the operating system	Embedded C compilers are OS independent
In C language, the standard compilers are used for executing a program	In embedded C language, specific compilers are used.
The popular compilers used in this language are GCC, Borland turbo C, Intel C++, etc	The popular compilers used in this language are Keil, BiPOM Electronics & green hill
The format of C language is free-format	Its format mainly depends on the kind of microprocessor used.
Optimization of this language is normal	Optimization of this language is a high level
It is very easy to modify & read	It is not easy to modify & read
Bug fixing is easy	Bug fixing of this language is complicated

Advantages

The **advantages of embedded c programmin**g include the following.

- It is very simple to understand.
- It executes a similar task continually so there is no requirement for changing hardware like additional memory otherwise storage space.
- It executes simply a single task at once
- The cost of the hardware used in the embedded c is typically so much low.
- The applications of embedded are extremely appropriate in industries.
- It takes less time to develop an application program.
- It reduces the complexity of the program.
- It is easy to verify and understand.
- It is portable from one controller to another.

Disadvantages

The **disadvantages of embedded c programming** include the following.

- At a time, it executes only one task but can't execute the multi-tasks
- If we change the program then need to change the hardware as well
- It supports only the hardware system.
- It has a scalability issue
- It has a restriction like limited memory otherwise compatibility of the computer.

Applications of Embedded C Program

The **applications of embedded c programming** include the following.

- Embedded C programming is used in industries for different purposes
- The programming language used in the applications is speed checker on the highway, controlling of traffic lights, controlling of street lights, tracking the vehicle, artificial intelligence, home automation, and auto intensity control.

Conclusion:

MPLAB X Programming IDE.

Step1: Creating a new project

- Go to the File Tab.
- Click on New Project.

Step1: Choose Project:

- Select: Microchip Embedded -> Standalone Project. Click Next.

Step2: Select Device:

- Select: Family -> Advanced 8 Bit MCU (PIC18).
- Select: Device: PIC18F4550

Step3: Select Tool: Simulator. Click Next.
Step4: Select Compiler ->XC8. Click Next.
Step5: Select Project Name and Folder.

- Give Project Name.
- Uncheck Set as main project option.
- Click Finish..

Step6: Creating a new Source file and Header File.

- Go to the Project location in the Project window.
- Click the + sign to open the project space.
- Right Click on the Source Files folder (for a C file) and Header files (for a .h file).
- New - > C main File/C Source file

Step7: Opening an existing project.

- Go to the File Tab.
- Select Open Project.

Experiment No. 2

Title : Write an Embedded C program to add array of n numbers.

Aim : To perform addition of array of n numbers using Embedded C programming language
Objectives : 1. To understand the Embedded C programming
2. To study corresponding instructions used to perform the array addition of n numbers.
Outcome : After performing this experiment student will be able to –
1. Able to write embedded C program
Apparatus/ Software required: MPLAB X Programming IDE
Theory:

Array

An array is defined as the collection of similar type of data items stored at contiguous memory locations. Arrays are the derived data type which can store the primitive type of data such as int, char, double, float, etc. It also has the capability to store the collection of derived data types, such as pointers, structure, etc. The array is the simplest data structure where each data element can be randomly accessed by using its index number.
An array is one special case of a pointer. A pointer points toward... something. It may point to a single type such as a char or an integer, even another pointer.
An array is actually also a pointer which points to the first member of a group of like types.
Pointers hold a specific memory address. If you add to a pointer it doesn't count that as memory locations but the size of the type it points toward.

In PIC18F each port has three registers for its operation. These registers are
• TRIS register (data direction register)
• PORT register (reads the levels on the pins of the device)
• LAT register (output latch)

1.TRIS REGISTER
TRIS is a data direction register. Setting TRIS bit for corresponding port will let know the data direction (whether read or write) to microcontroller. Each PORT has its own TRIS register E.g: For PORT A

```
TRISA=0     //making port as output port (write)
TRISA=1     //making port as input port (read)
```

We can also make specific bits of port as input or output.

```
TRISA.F2 = 1;                 // PORTA pin RA2 configured as input
```

2.PORT REGISTER
It reads the levels on the pins of the device and it assigns logic values (0/1) to the ports. The role of the PORT register is to receive the information from an external source (like a sensor) or to send information to the external elements (like an LCD). E.g: For input mode of PORT A

Reading the PORTA register reads the status of the pins. We will firstly set the direction of data by TRIS register TRISA bit=1. It will make the corresponding PORTA pin an input and put the corresponding output driver in a Hi-Impedance mode.

TRISA=1 //making port as input port (read)
PORTA=0x** //Assigning low logic to the pins by external circuitry or device (Push button)

For output mode of PORT A

We will first set the direction of data by TRIS register. Then port is given the value for output. A write to the PORT register writes the data value to the port latch.

TRISA=0 //making port as output port (write)
PORTA=0x03; //Assigning high logic to the RA0 and RA1

3.LAT REGISTER

The Data Latch register is also memory mapped. LAT register is associated with an I/O pin. It eliminates the problems that could occur with read-modify-write instructions.

Read-modify-write instructions.

These are those operations that first read the port make the necessary operations(e.g. AND,OR)and then write back the port.

Latch Read

A read of the LAT register returns the values held in the port output latches, instead of the values on the I/O pins. A read-modify-write operation on the LAT register which is associated with an I/O port, avoids the possibility of writing the input pin values into the port latches.

Latch Write

A write to the LAT register has the same effect as a write to the PORT register. A write to the PORT register writes the data value to the port latch. Similarly, a write to the LAT register writes the data value to the port latch.

In 18FXXX devices the LAT register holds the value written to the PORT irrespective of the voltage level on the PORT pins. The actual logic level on the port pins can be read through PORT register.

Logic to find sum of array elements

1. Input size and elements in array, store in some variable say n and number[n].
2. To store sum of array elements, initialize a variable sum = 0. **Note:** sum must be initialized only with 0.
3. To find sum of all elements, iterate through each element and add the current element to the sum. Which is run a loop from 0 to n. The loop structure should look like for(i=0; i<n; i++).
4.Inside the loop add the current array element to sum i.e. sum = sum + number [i] or even you can do sum += number [i]

Conclusion :

Experiment No. 3

Title : Write an Embedded C program for Internal to internal memory transfer

Aim : To perform transfer of elements from internal to internal memory location using Embedded C programming language

Objectives : 1. To understand the Embedded C programming
2. To study corresponding instructions used to perform transfer of elements internal to internal memory location.

Outcome : After performing this experiment student will be able to –
1. Able to write embedded C program for Internal to internal memory transfer

Apparatus/ Software required: MPLAB X Programming IDE

Theory:

The PIC18 Memory Organization –

Data Memory and Program Memory are separated Data Memory and Program Memory are separated Separation of data memory and program memory makes possible the simultaneous access of data and instruction.

Data memory are used as general-purpose registers or special function registers

On-chip p Data EEPROM are provided in some PIC18 MCUs

PIC18F Memory types

Data Memory

Program Memory

Stack Memory

Data Memory

- Used for transitory data when the program is being executed
  Example: A=1, B=2, C=3 X=A+B+Cà A+B=W;W+C=W
- Data memory is either SRAM or EEPROM. n Some chips only have SRAM n Others may have SRAM and EEPROM o EEPROM stores permanently
- Various PIC18 versions contain between 256 and 3968 bytes of data memory
  For example: SRAM data memory begins at 12-bit address 0x000 and ends at 12-bit address 0xFFF (4K)
- Data memory is often divided into two sections
  General Function Registers (GFR) or register file location -000-0xF7F locations
  Special Function Registers (SFR) – specific to PIC - 0xF80-0xFFF (upper 128 bytes)
- Depending on the PIC chip, the sizes for GFR and SFR are different

Program Memory

- Program memory is 16-bits wide accessed through a separate program data bus and address bus inside the PIC18.
- Program memory stores the program and also static data in the system.
  On-chip n External
  On-chip program memory is either PROM or EEPROM.
- The PROM version is called OTP (one-time programmable) (PIC18C)
- The EEPROM version is called Flash memory (PIC18F).
- Maximum size for program memory is 2M
- Program memory addresses are 21-bit address starting at location 0x000000

Program Stack Memory Saving the return address

- The PIC18 contains a program stack that stores up to 31 return addresses from functions.
  31-deep
  The program stack is 21 bits in width as is the program address (remember address memory is 2M)
- Stack memory uses SRAM o Operation of a stack n When a function is called, the return address (location of the next step in a program) is pushed onto the stack.
  For example: Stack number 1 will have value= 0x0x1F0000
- When the return occurs within the function, the return address is retrieved from the stack and placed into the program counter.

Algorithm

1. Input size and elements in array, store in some variable say n and source1[n]
2. To store the transferred elements initialize destination[].initialize destination with 0
3. To transfer all elements from source1[], iterate through each element and transfer the current element to the destination. Which is run a loop from 0 to n. The loop structure should look like for(i=0; i<n; i++).
4. Inside the loop assign the current array element to destination i.e dest[i] = source1[i]; This instruction act as MOV instruction.

Conclusion :

Experiment No. 4

Title : Write an Embedded C menu driven program for :
i) Multiply 8 bit number by 8 bit number
ii) Divide 8 bit number by 8 bit number

Aim : To perform multiplication and division for 8 bit no using Embedded C programming language

Objectives : 1. To understand the Embedded C programming
2. To study corresponding instructions used to perform perform multiplication and division for 8 bit no

Outcome : After performing this experiment student will be able to –
1. To write embedded C program for multiplication and division for 8 bit no

Apparatus/ Software required: MPLAB X Programming IDE

Theory:

i) Multiply 8 bit number by 8 bit number

Algorithm

1. Input two 8 bit no in two variables num1 and num2
2. To store the result of multiplication initialize variable result. result must be initialize with 0
3. To find result of multiplication of two 8 bit nos using successive addition method, iterate num1 and add the num1 to the sum. Which is run a loop from 0 to num2. The loop structure should look like for(i=1; i<=num2; i++).
4. Inside the loop add the num1 to sum i.e. in result = result + num1
5. To see the output on PORTB set
   TRISB=0 //initialize Port_B as output
   PORTB = result; // from sum to PORT_B

Conclusion:

Experiment No. 5

Title : Write an Embedded C program for sorting the numbers in ascending and descending order.

Aim : To perform sorting the numbers in ascending and descending order using Embedded C programming language

Objectives : To understand the Embedded C programming

1. To study corresponding instructions used to sorting the numbers in ascending and descending order

Outcome : After performing this experiment student will be able to –

1. To write embedded C program for sorting the numbers in ascending and descending order

Apparatus/ Software required: MPLAB X Programming IDE

Theory:

Algorithm

Ascending Order

1. Start at index zero, compare the element with the next one (a[0] & a[1] (a is the name of the array)), and swap if a[0] > a[1]. Now compare a[1] & a[2] and swap if a[1] > a[2]. Repeat this process until the end of the array. After doing this, the largest element is present at the end. This whole thing is known as a pass. In the first pass, we process array elements from [0,n-1].
2. Repeat step one but process array elements [0, n-2] because the last one, i.e., a[n-1], is present at its correct position. After this step, the largest two elements are present at the end.
3. Repeat this process n-1 times

Descending Order

1. Start at index zero, compare the element with the next one (a[0] & a[1] (a is the name of the array)), and swap if a[0] < a[1]. Now compare a[1] & a[2] and swap if a[1] <a[2]. Repeat this process until the end of the array. After doing this, the largest element is present at the end. This whole thing is known as a pass. In the first pass, we process array elements from [0,n-1].
2. Repeat step one but process array elements [0, n-2] because the last one, i.e., a[n-1], is present at its correct position. After this step, the largest two elements are present at the end.
3. Repeat this process n-1 times

Conclusion: